Interactive visualizations

Visualization of Information

Salomon Eisler

Dept. of Industrial Engineering and Intelligent Systems

Agenda

1. Why?
2. PowerBi
3. Using AI
4. Structure of a Shiny App - R
5. Layout
6. Widgets
7. Use R scripts and data
8. Examples & Reactive Expressions

Why Interactions Matter in Visualizations

1. Promote Active Exploration: Users engage more deeply by manipulating views, not just observing them.
2. Supports Curiosity and Information Foraging: Dynamic exploration can reveal patterns and anomalies that static views may miss.
3. Adapt to User Goals: Interactions let users tailor the visualization to their specific questions and tasks.
4. Support Iterative Thinking: Users refine their understanding through trial, error, and continuous refinement.
5. Enhance Trust and Engagement: Interactive systems feel responsive and user-centered, boosting confidence in the data.

Interaction Support

1. Select – Mark something as interesting
2. Explore – Show me something else
3. Reconfigure – Show me a different arrangement
4. Encode – Show me a different representation
5. Abstract/Elaborate – Show me more or less detail
6. Filter – Show me something conditionally
7. Connect – Show me related items
8. Undo/Redo – Let me go to where I have already been
9. Change configuration – Let me adjust the interface

Many options

1. R
2. Python
3. PowerBI
4. Tableau
5. Excel
6. Cognos
7. AI
8. …

What has changed with AI?

- AI tools generate charts/dashboards from plain natural language prompts.
- Less need to memorize Python/R syntax.
- AI auto-cleans data and fixes code errors.
- MCP-connected AI can query data sources directly.
- Focus is shifting from coding skills to analytical thinking.

MCP

- MCP stands for Model Context Protocol.
- Standard that lets AI models connect to external tools and data sources safely and dynamically.
- Allows AI tools like Claude Code or OpenAI agents to:
  - access databases
  - read files
  - call APIs
  - use GitHub
  - query Whatsapp/Slack/Jira
  - interact with development tools automatically.

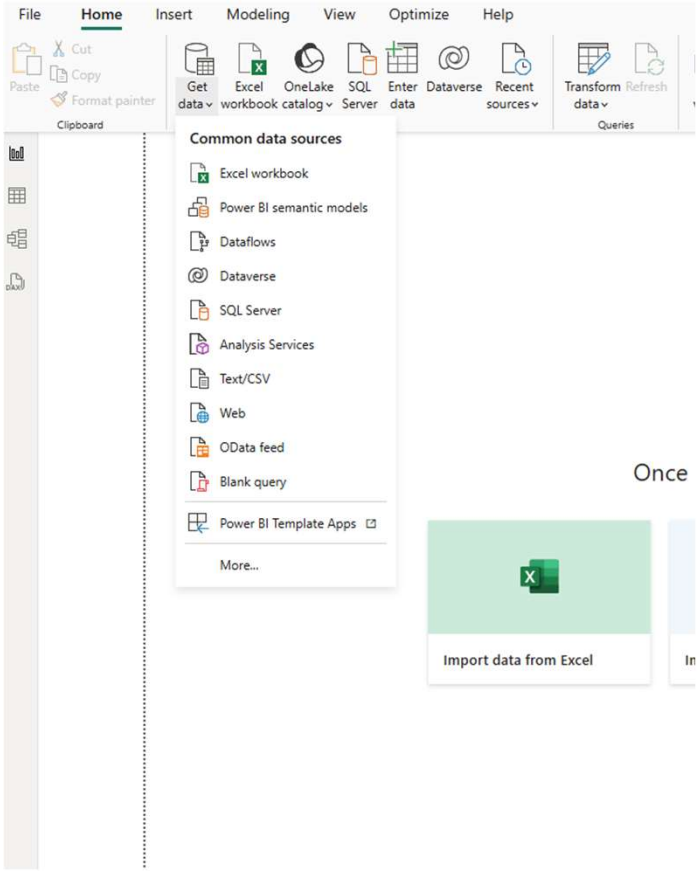
PowerBI

Building blocks of Power BI – low code

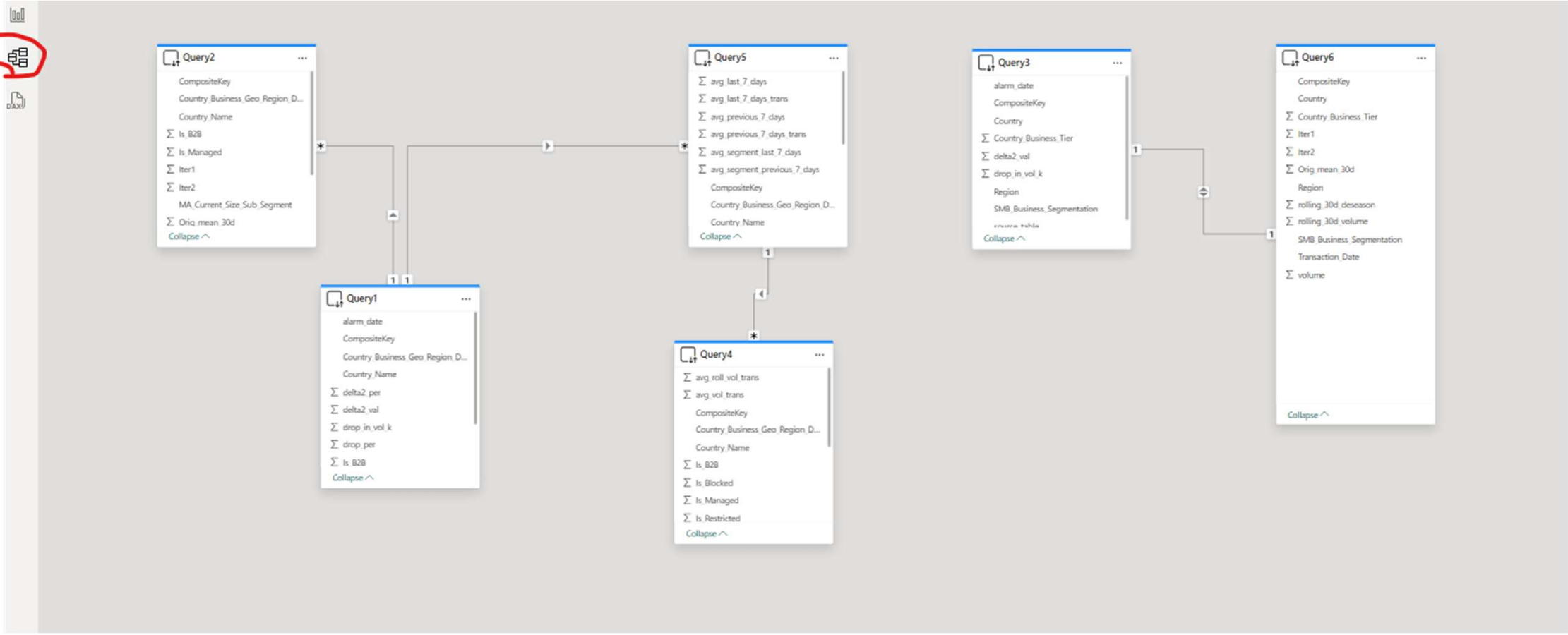
1. Create a semantic model
   - consists of all connected data, transformations, relationships, and calculations.
2. Create visualizations:
   - add them to the canvas for a report page.
   - drag and drop
   - Link/interactivity between visuals
3. Publish report to Power BI service.

Source: https://learn.microsoft.com/en-us/training/modules/get-started-with-power-bi/?WT.mc_id=powerbi_home_inproduct_introcard

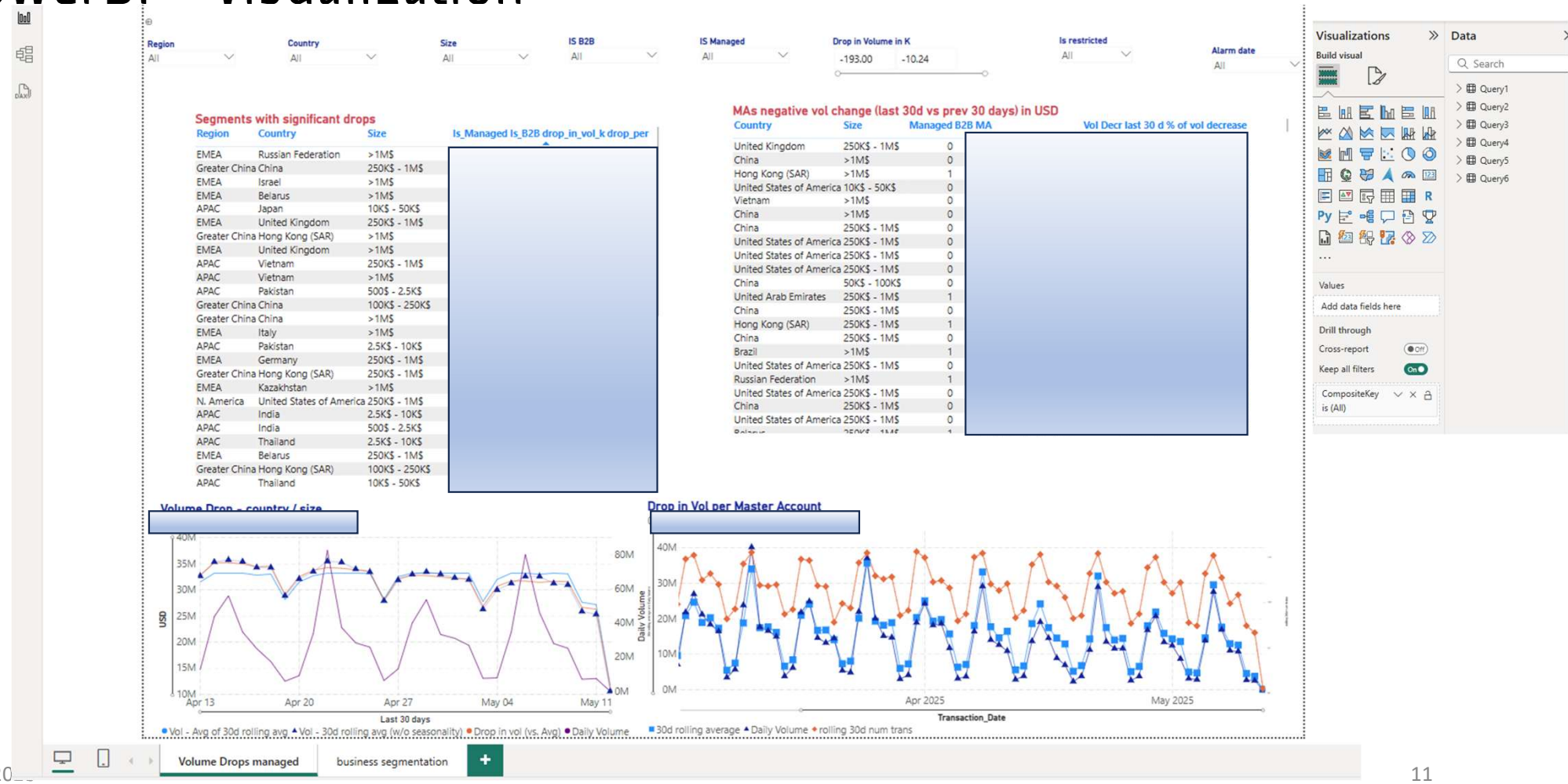
PowerBI – get data



PowerBI – semantic model

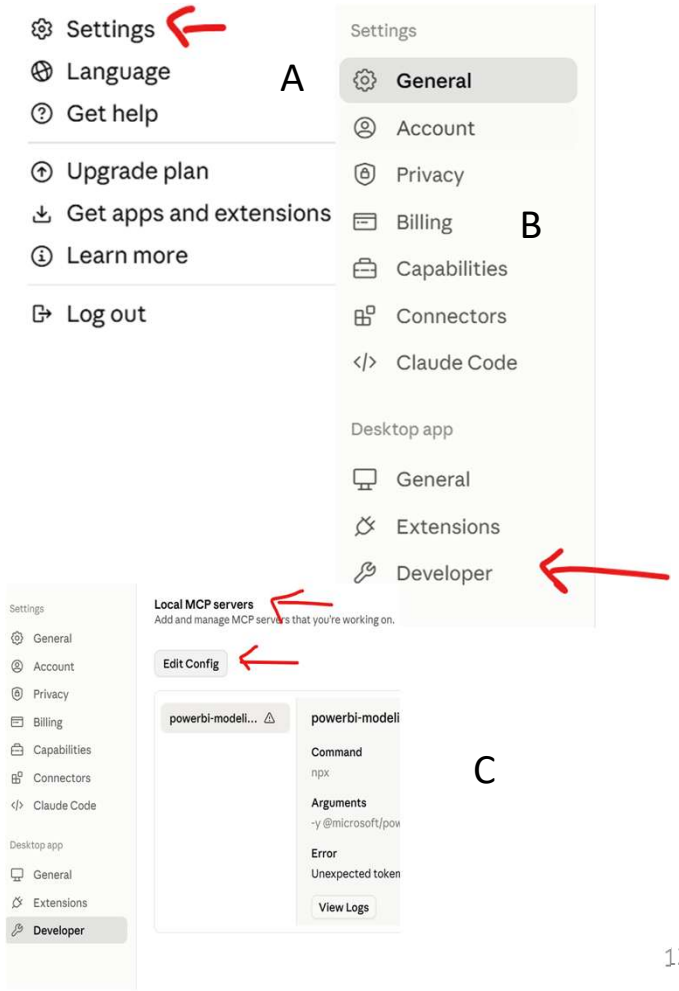


PowerBI – visualization



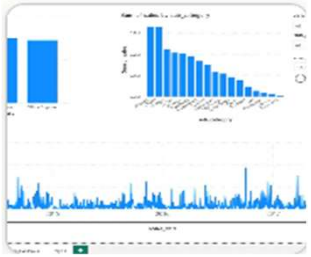
PowerBI and MCP to build a dashboard using Python

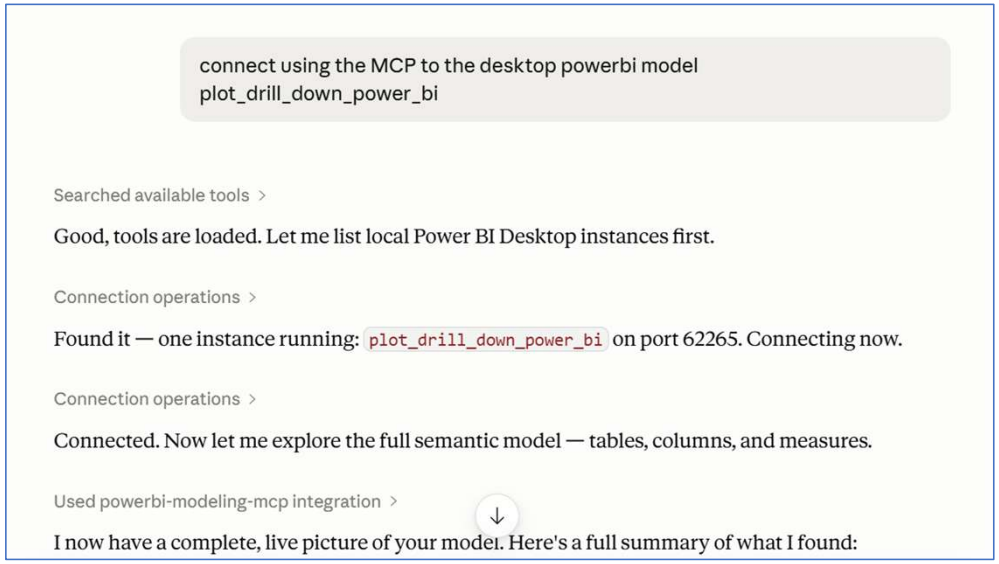
1. Install the powerbi-modelling-mcp from: https://github.com/microsoft/powerbi-modeling-mcp
2. Install the Claude app in your desktop
3. Go to the Developer and edit the file claude_desktop_config.json to include the code from the ms github
4. Then open again Claude
5. Then open your PowerBI dashboard – or just the semantic model if you do not have a dashboard



PowerBI and MCP to build a dashboard using Python

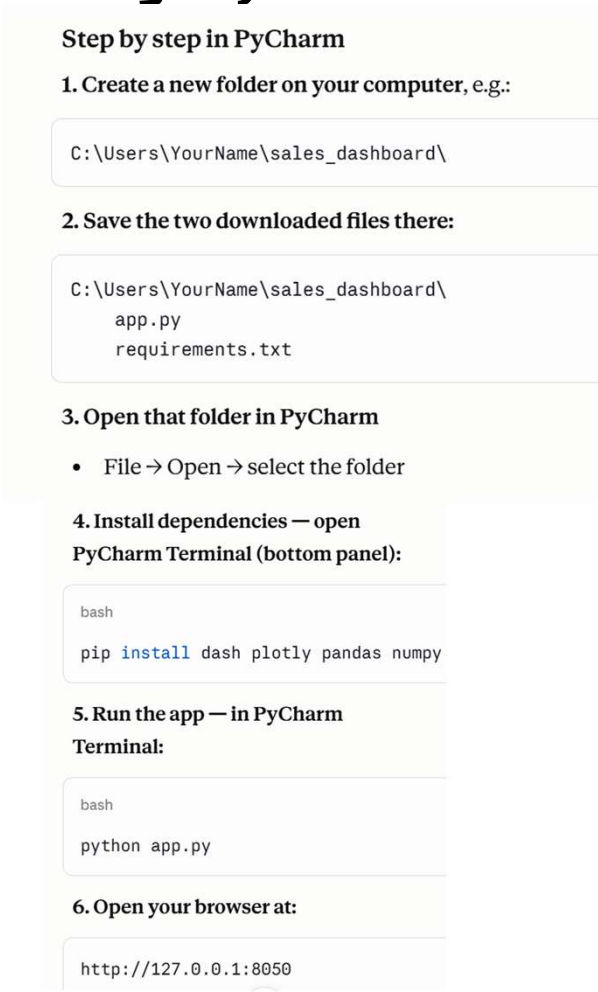
1. Open a new prompt in Claude
2. Then enter in the prompt: provide me with Python code to run locally which will provide a dashboard similar to the image here (which was done using PowerBI) on the same semantic model which you just connected.





PowerBI and MCP to build a dashboard using Python

1. In Python : pip install dash plotly pandas numpy
2. Then enter in the prompt: provide me with Python code to run locally which will provide a dashboard similar to the image here (which was done using PowerBI) on the same semantic model which you just connected.



Step by step in PyCharm

1. Create a new folder on your computer, e.g.:

```
C:\Users\YourName\sales_dashboard\
```

2. Save the two downloaded files there:

```
C:\Users\YourName\sales_dashboard\
    app.py
    requirements.txt
```

3. Open that folder in PyCharm

- File → Open → select the folder

4. Install dependencies — open PyCharm Terminal (bottom panel):

```bash
pip install dash plotly pandas numpy
```

5. Run the app — in PyCharm Terminal:

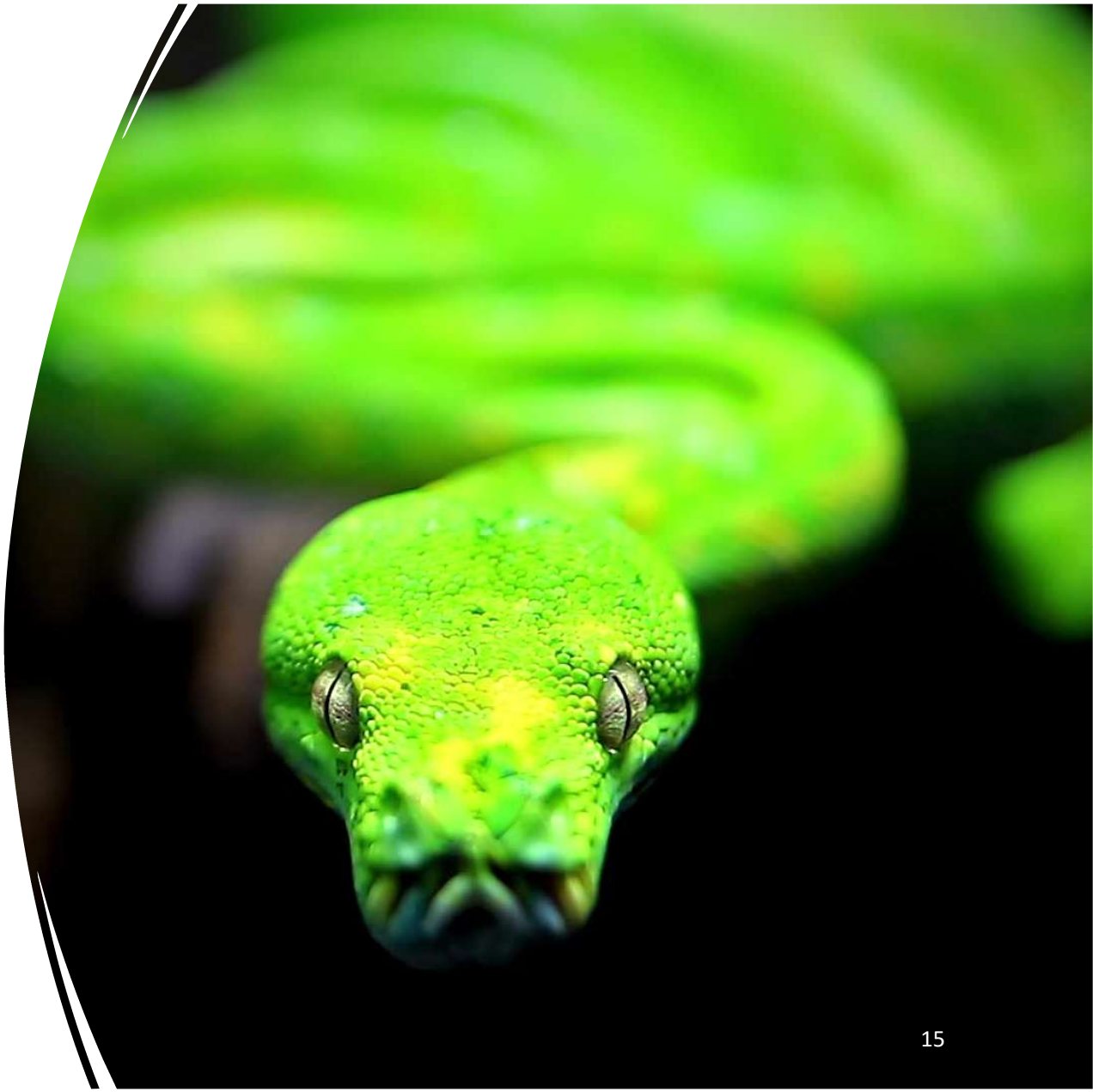
```bash
python app.py
```

6. Open your browser at:

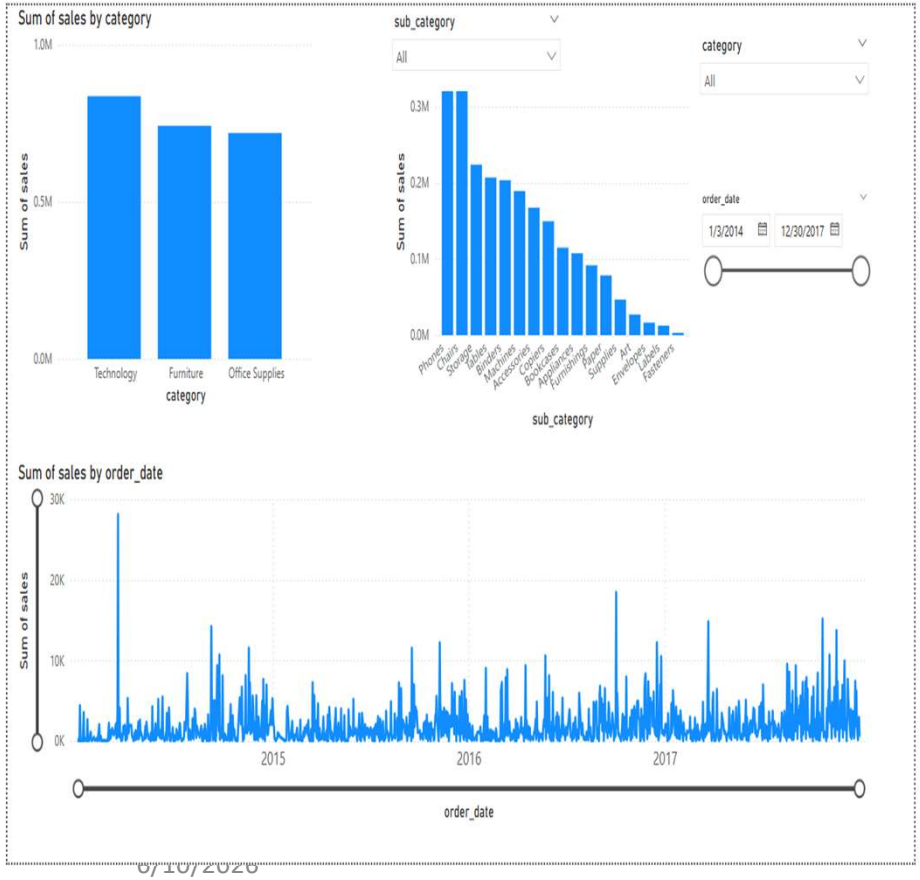
```
http://127.0.0.1:8050
```

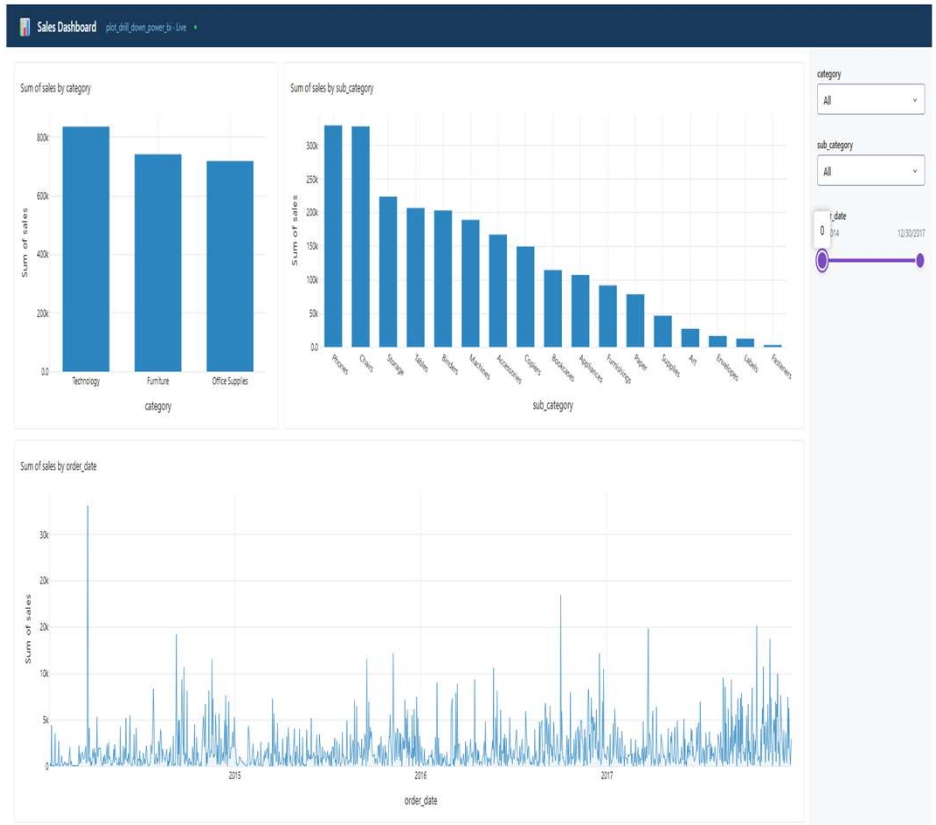
Why it is important to use the MCP to connect to the semantic model?

1. In Python : pip install dash plotly pandas numpy
2. Then enter in the prompt: provide me with Python code to run locally which will provide a dashboard similar to the image here (which was done using PowerBI) on the same semantic model which you just connected.
3. In Pycharm terminal: python app.py



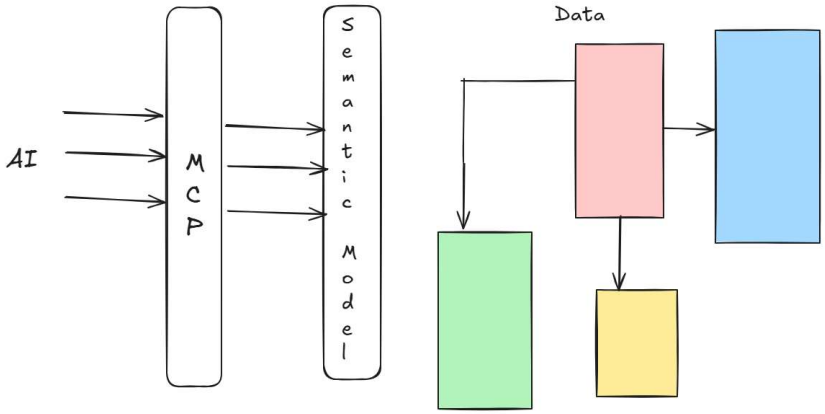
PowerBI and MCP to build a dashboard using Python





Why AI Needs Access to the Semantic Model

1. Trust needs Data Governance
2. When AI generates code based on the semantic model, every developer on the team gets code that references the same single source of truth — preventing the proliferation of inconsistent KPI definitions across dashboards
3. Without this, different students/developers may build dashboards where "Total Sales" means different things — a classic data governance failure
4. AI generating code without semantic model access is like a new employee writing reports without reading the data dictionary — the output may look correct but contain fundamental logical errors that only surface when business decisions are made on wrong numbers



Without a Semantic Model (raw databases, CSV, APIs, etc.)

You would have to manually provide the AI (e.g. Claude) with everything the model would have given automatically:

1. Schema description — table names, column names, data types
2. Relationships — how tables join, on which keys, cardinality
3. Business logic — what each measure means, how KPIs are calculated
4. Grain definition — what one row represents in each table
5. Known gotchas — columns that look numeric but are IDs, date formats, nulls

This is done either in the **prompt** ("here is my schema…") or in a **skill/system prompt** that persists across conversations.

The Core Difference

	Semantic Model	Manual Description
Setup effort	One-time MCP connection	Every conversation
Risk of error	Very low	High — human can forget details
Stays up to date	Automatic	Must be manually maintained
Governance	Built in	Depends on the prompt author

This is actually one of the strongest arguments for investing in a proper semantic layer — it becomes the **single interface** not just for humans and BI tools, but also for AI, ensuring everyone including Claude/Open AI/etc are working from the same governed definition of the data.

The Most Common Mistakes When Prompting AI for Visualizations

Mistake	Consequence
Not specifying audience	Wrong level of detail
Not specifying interactivity	Wrong technology chosen
Not specifying color/branding	Generic look that needs rework
Not specifying data volume	Slow or crashing dashboard
Not specifying number formats	Raw numbers instead of $1.2M
Assuming AI knows your business terminology	Wrong chart titles and labels

The more context you give AI upfront — audience, purpose, data volume, interactivity, branding — the less rework you do afterward. A well-specified prompt for a dashboard is almost like writing a functional specification — it pays to invest 10 minutes thinking it through before asking.

What AI Does vs What BI Tools Do

Capability	Tableau / Cognos / Power BI	AI (Claude, Copilot, etc.)
Interactive drag & drop	Native	No
Pixel-perfect formatting	Native	Partial
Enterprise security & governance	Mature	Depends on setup
Natural language querying	Limited	Native
Code generation	No	Native
Explaining what the data means	No	Native
Automated insight detection	Limited	Native
Self-service for non-technical users	Yes	Still evolving

The More Accurate Picture — Three Scenarios

1. Scenario 1: AI as an Accelerator for BI Tools
   - The most common scenario today. AI helps you work faster inside existing tools
2. Scenario 2: AI Replacing Simple Reports
   - For straightforward, code-generated dashboards like the one we built today — AI can produce a working Dash or Shiny dashboard without a BI tool at all.
3. Scenario 3: Conversational BI — The Emerging Frontier
   - Instead of building a dashboard at all, users just ask questions in natural language.

What the BI Vendors Are Doing

They are not standing still, all major vendors are embedding AI heavily:

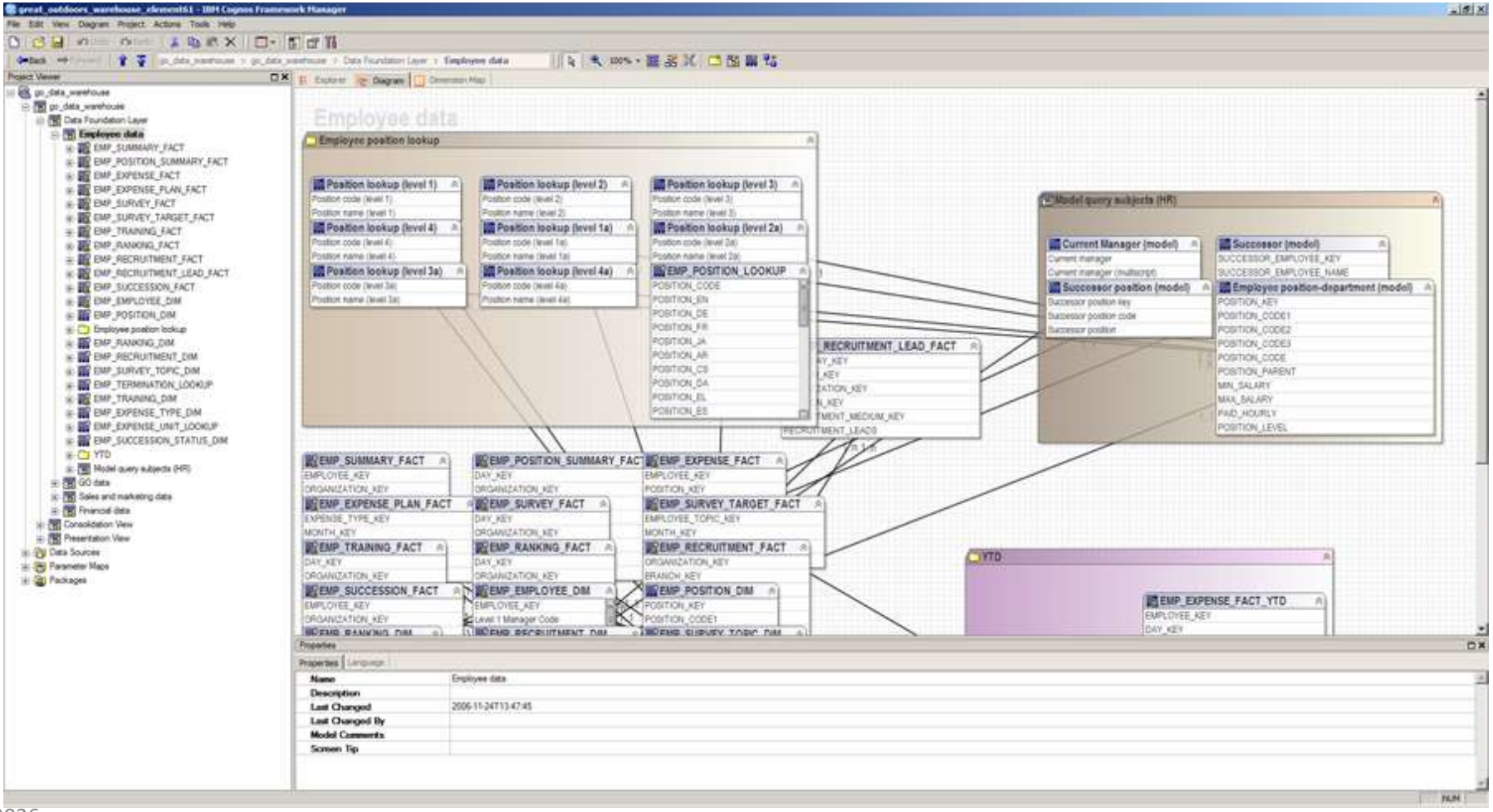
- **Power BI** → Copilot generates reports, writes DAX, explains visuals
- **Tableau** → Tableau AI, Einstein integration, natural language queries
- **Cognos** → IBM watsonx integration
- **Looker** → Gemini integration from Google
- **Qlik** → Qlik AI with AutoML

The BI tools are essentially **absorbing AI capabilities** to stay relevant, which means the tools themselves are evolving rather than disappearing.

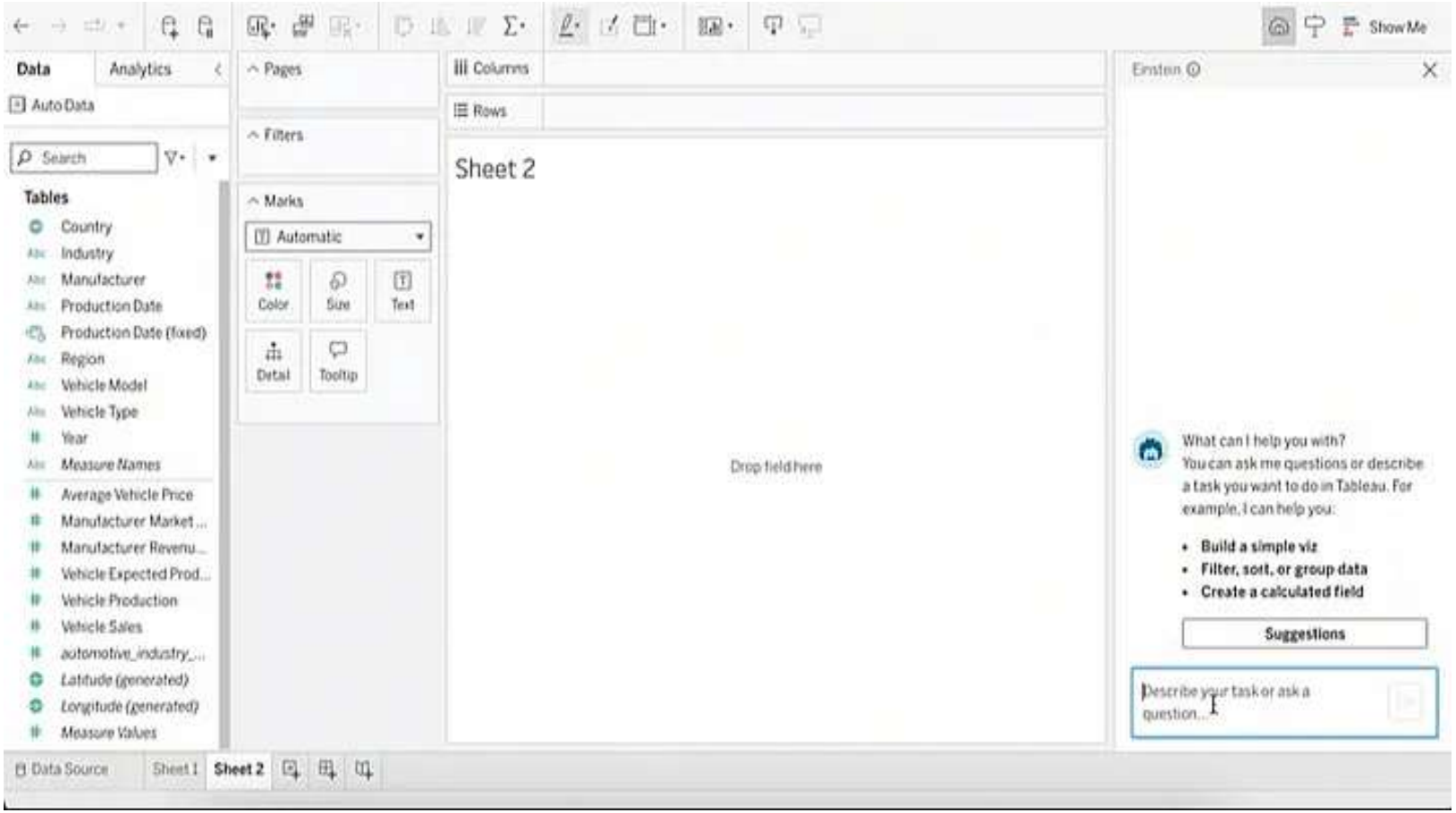
What is the role of a human BI developer when AI can generate a dashboard from a business question in seconds?

- Less time on formatting and DAX syntax or writing code
- More time on deciding which visualizations/interactions to use
- More time on data modeling and semantic layer design
- More time on asking the right business questions
- More time on data governance and trust
- More time on interpreting and communicating results

Examples of Semantic Layers - Cognos



Examples of Semantic Layers - Tableau

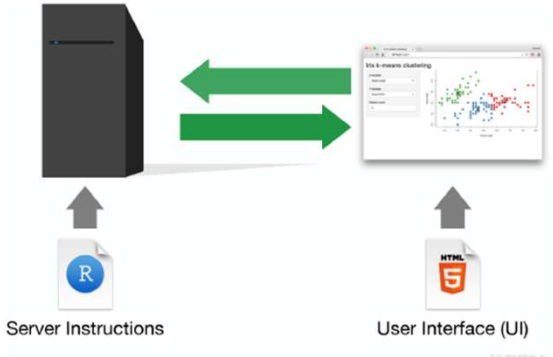


Shiny R

- Shiny is an R package that makes it easy to build interactive web applications (apps) straight from R. (https://shiny.posit.co/)
- It also exists in Python (https://shiny.posit.co/py/)

Structure of a Shiny App

- Shiny apps are contained in a single script called app.R.
- The script app.R lives in a directory (for example, newdir/) and the app can be run with runApp("newdir").
- app.R has three components:
  - ✓a user interface object (ui)
    - controls the layout and appearance of the app
    - nested R functions that assemble an HTML user interface for the app
  - ✓a server function (server)
    - contains instructions on how to build and rebuild the R objects displayed in the UI
  - ✓a call to the shinyApp function
    - combines ui and server into a functioning app
    - creates Shiny app objects from an explicit UI/server pair



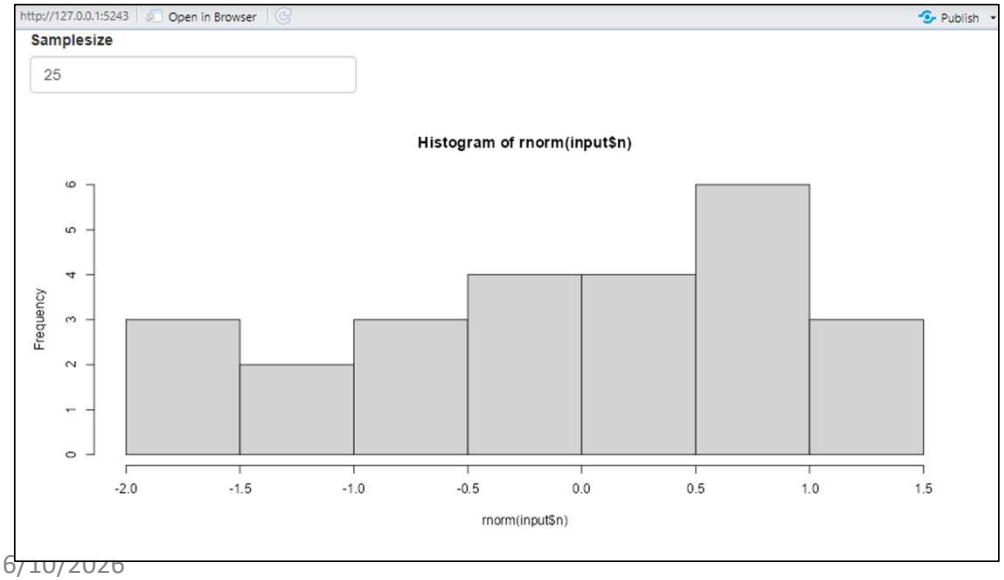
Structure of a Shiny App

```
library(shiny)

# Define UI ----
ui <- fluidPage(
  )
# Define server logic ----
server <- function(input, output) {
}
# Run the app ----
shinyApp(ui = ui, server = server)
```

Simple example

- Note the 3 components.
- In the UI part the functions that are defined in the server part are called.

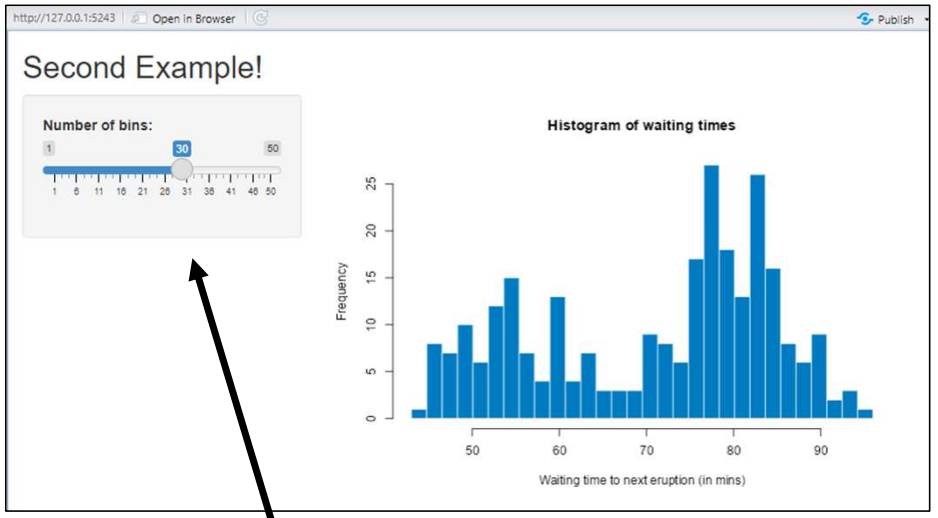



```
library(shiny)
ui<-fluidPage(
numericInput(inputId="n",
"Samplesize",value=25),
plotOutput(outputId="hist"))
server<-function(input,output){
output$hist<-renderPlot({
hist(rnorm(input$n))
})}
shinyApp(ui=ui,server=server)
```

simple_example

Second Example - widgets

```
library(shiny)
ui <- fluidPage(
titlePanel("Second Example!"),
sidebarLayout(
sidebarPanel(
sliderInput(inputId = "bins", label = "Number of bins:", min = 1, max = 50, value = 30)
   ),
      mainPanel(
plotOutput(outputId = "distPlot")
   )
  )
)
server <- function(input, output) {
# This expression that generates a histogram is wrapped in a call to renderPlot to indicate that:
 # 1. It is "reactive" and therefore should be automatically  re-executed when inputs (input$bins) change
 # 2. Its output type is a plot
 output$distPlot <- renderPlot({
   x    <- faithful$waiting
   bins <- seq(min(x), max(x), length.out = input$bins + 1)
   hist(x, breaks = bins, col = "#007bc2", border = "white",
        xlab = "Waiting time to next eruption (in mins)",
        main = "Histogram of waiting times")
 })
}
shinyApp(ui = ui, server = server)
```



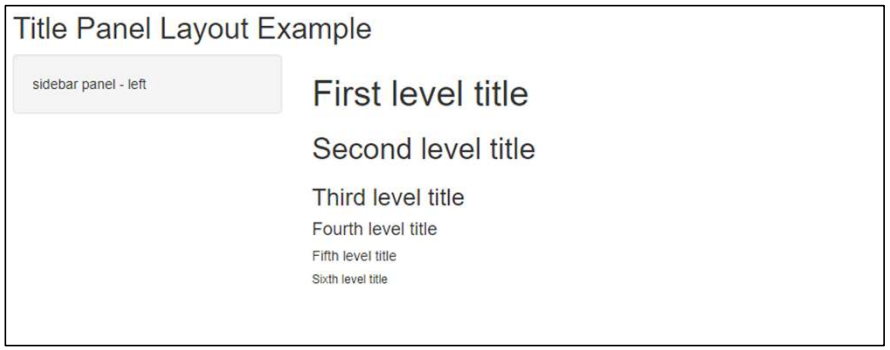
widget

other_input_widgets

Layout

- function fluidPage to create a display that automatically adjusts to the dimensions of the user's browser window

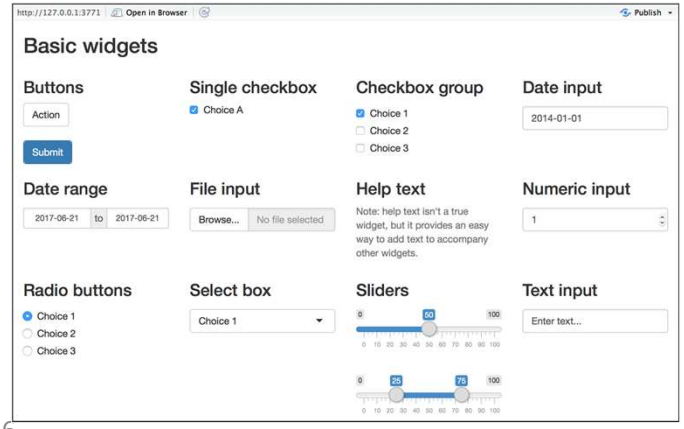
```
ui <- fluidPage(
  titlePanel("Title Panel Layout Example"),
  sidebarLayout(position = "left",
    sidebarPanel("sidebar panel - left"),
    mainPanel(
      h1("First level title"),
      h2("Second level title"),
      h3("Third level title"),
      h4("Fourth level title"),
      h5("Fifth level title"),
      h6("Sixth level title")
    )
  )
)
```



layout

Control Widgets

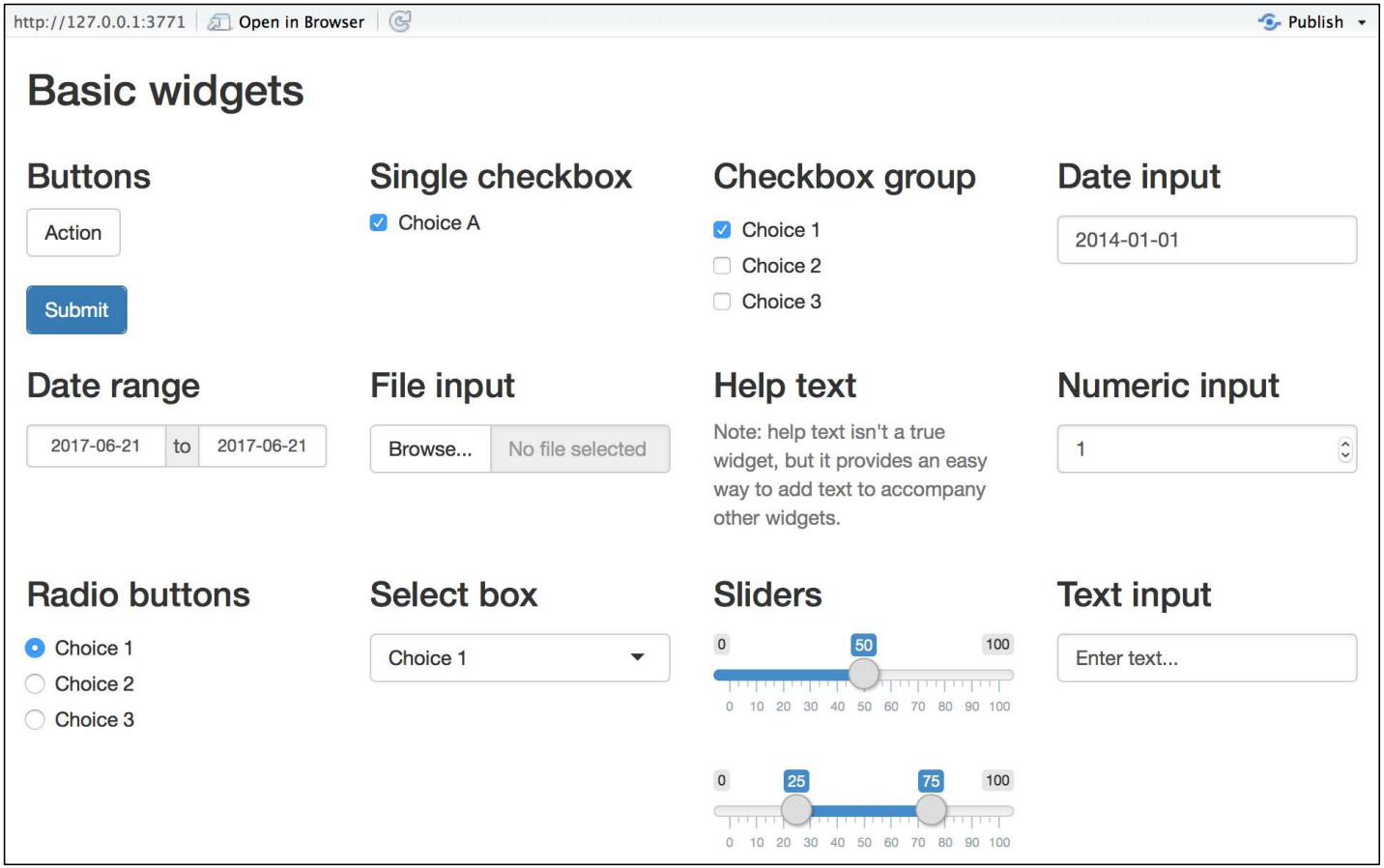
- Widget: A web element that users can interact with. Widgets provide a way for users to send messages to the Shiny app
- Shiny widgets collect a value from your user. When a user changes the widget, the value will change as well.



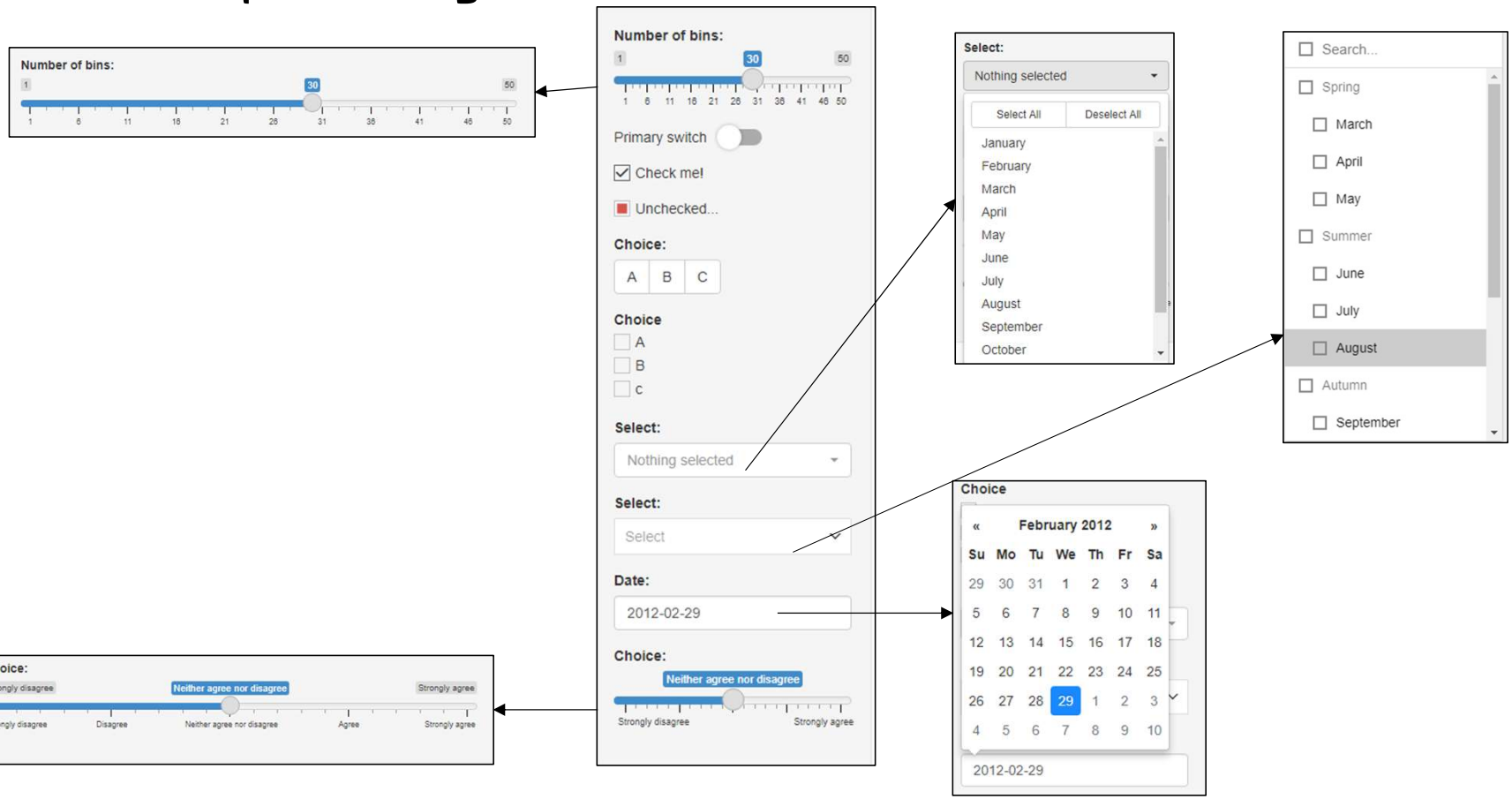
function	widget
actionButton	Action Button
checkboxGroupInput	A group of check boxes
checkboxInput	A single check box
dateInput	A calendar to aid date selection
dateRangeInput	A pair of calendars for selecting a date range
fileInput	A file upload control wizard
helpText	Help text that can be added to an input form
numericInput	A field to enter numbers
radioButtons	A set of radio buttons
selectInput	A box with choices to select from
sliderInput	A slider bar
submitButton	A submit button
textInput	A field to enter text

control_widgets

Control Widgets



Other input widgets



https://dreamrs.github.io/shinyWidgets/
https://shiny.rstudio.com/reference/shiny/latest/dateinput

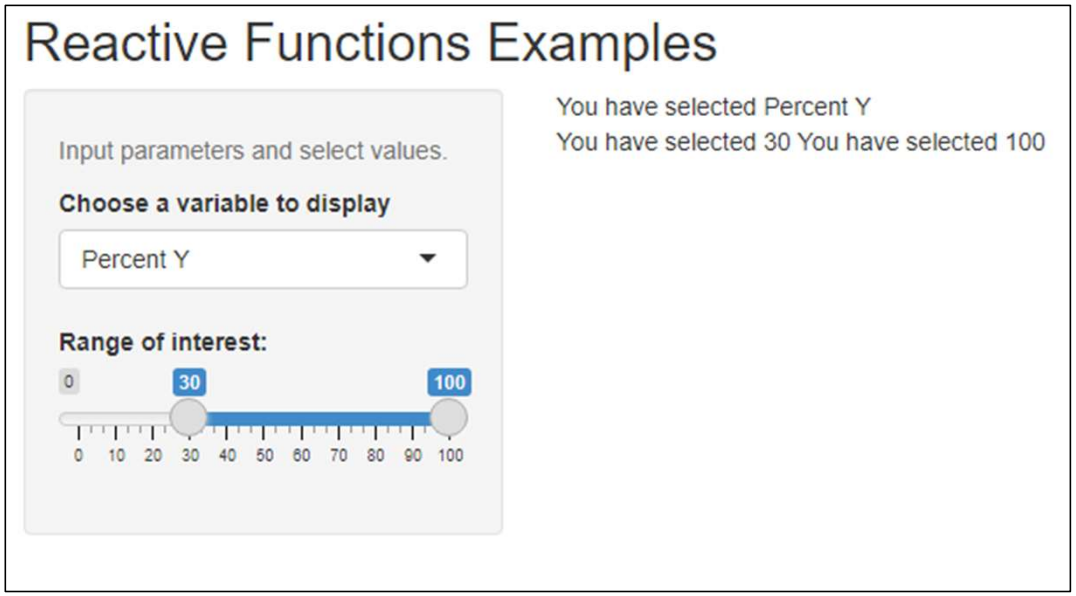
Reactiveness - Display reactive output

The ability of to automatically update the output displayed to the user in response to changes made to the input

- Automatic Output Updates:
  - Outputs respond instantly to changes in input without manual refresh or re-running code.
- Reactive Data Flow
  - Shiny tracks dependencies between inputs and outputs, updating only what's needed.
- Improved User Experience
  - Users get immediate feedback, making the app feel dynamic and responsive.
- No Redundant Computation
  - Only the affected parts of the app update, keeping performance efficient.
- Foundation of Interactivity
  - Reactivity enables Shiny apps to behave like real-time interactive tools.

Display Reactive: simple example

```
ui <- fluidPage(
  titlePanel("Reactive Functions Examples"),
    sidebarLayout(
    sidebarPanel(
      helpText("Input parameters and select values."),
        selectInput("var",  label = "Choose a variable to display",  choices = c("Percent X",   "Percent Y",    "Percent Z",   "Percent W"),  selected = "Percent X"),
           sliderInput("range",     label = "Range of interest:",       min = 0, max = 100, value = c(0, 100))),
        mainPanel(
      textOutput("selected_var"),
      textOutput("selected_value")     ) ))
server <- function(input, output) {
    output$selected_var <- renderText({
    paste("You have selected", input$var)   })
  output$selected_value <- renderText({
    paste("You have selected", input$range)  })
}
shinyApp(ui = ui, server = server)
```



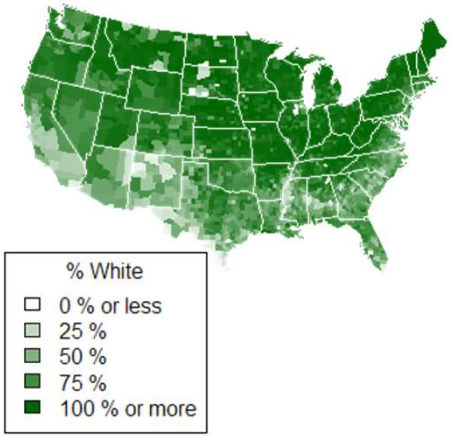
Use R scripts and data

- Will use counties.rds: dataset of demographic data for each county in the United States, collected with the UScensus2010 R package.
- The dataset in counties.rds contains
  - the name of each county in the United States
  - the total population of the county
  - the percent of residents in the county who are White, Black, Hispanic, or Asian

```
> head(counties)
             name total.pop white black hispanic asian
1 alabama,autauga     54571  77.2  19.3      2.4   0.9
2 alabama,baldwin    182265  83.5  10.9      4.4   0.7
3 alabama,barbour     27457  46.8  47.8      5.1   0.4
4    alabama,bibb     22915  75.0  22.9      1.8   0.1
5  alabama,blount     57322  88.9   2.5      8.1   0.2
6 alabama,bullock     10914  21.9  71.0      7.1   0.2
```

Use R scripts and data

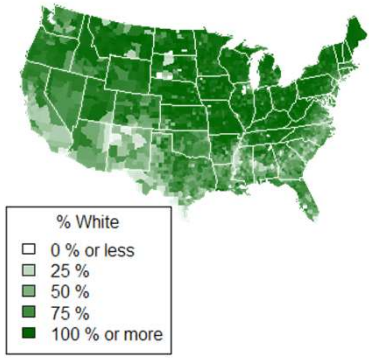
- helpers.R is an R script that can help you make choropleth maps
- A choropleth map is a map that uses color to display the regional variation of a variable.
- helpers.R will create percent_map, a function designed to map the data in counties.rds.



Wikipedia: A choropleth map (from Greek χῶρος (choros) 'area/region', and πλῆθος (plethos) 'multitude') is a type of statistical thematic map that uses pseudocolor, meaning color corresponding with an aggregate summary of a geographic characteristic within spatial enumeration units, such as population density or per-capita income.

https://shiny.rstudio.com/tutorial/written-tutorial/lesson5/census-app/helpers.R

Use R scripts and data



- The percent_map function in helpers.R takes five arguments:
  - var a column vector from the counties.rds dataset
  - color any character string you see in the output of colors()
  - legend.title A character string to use as the title of the plot's legend
  - max A parameter for controlling shade range (defaults to 100)
  - min A parameter for controlling shade range (defaults to 0)
- Use percent_map at the command line to plot the counties data as a choropleth map, like this.

```
library(maps)
library(mapproj)
source("census-app/helpers.R")
counties <- readRDS("counties.rds")
percent_map(counties$white, "darkgreen", "% White")
```

census

https://shiny.rstudio.com/tutorial/written-tutorial/lesson5/census-app/helpers.R

Use R scripts and data - UI

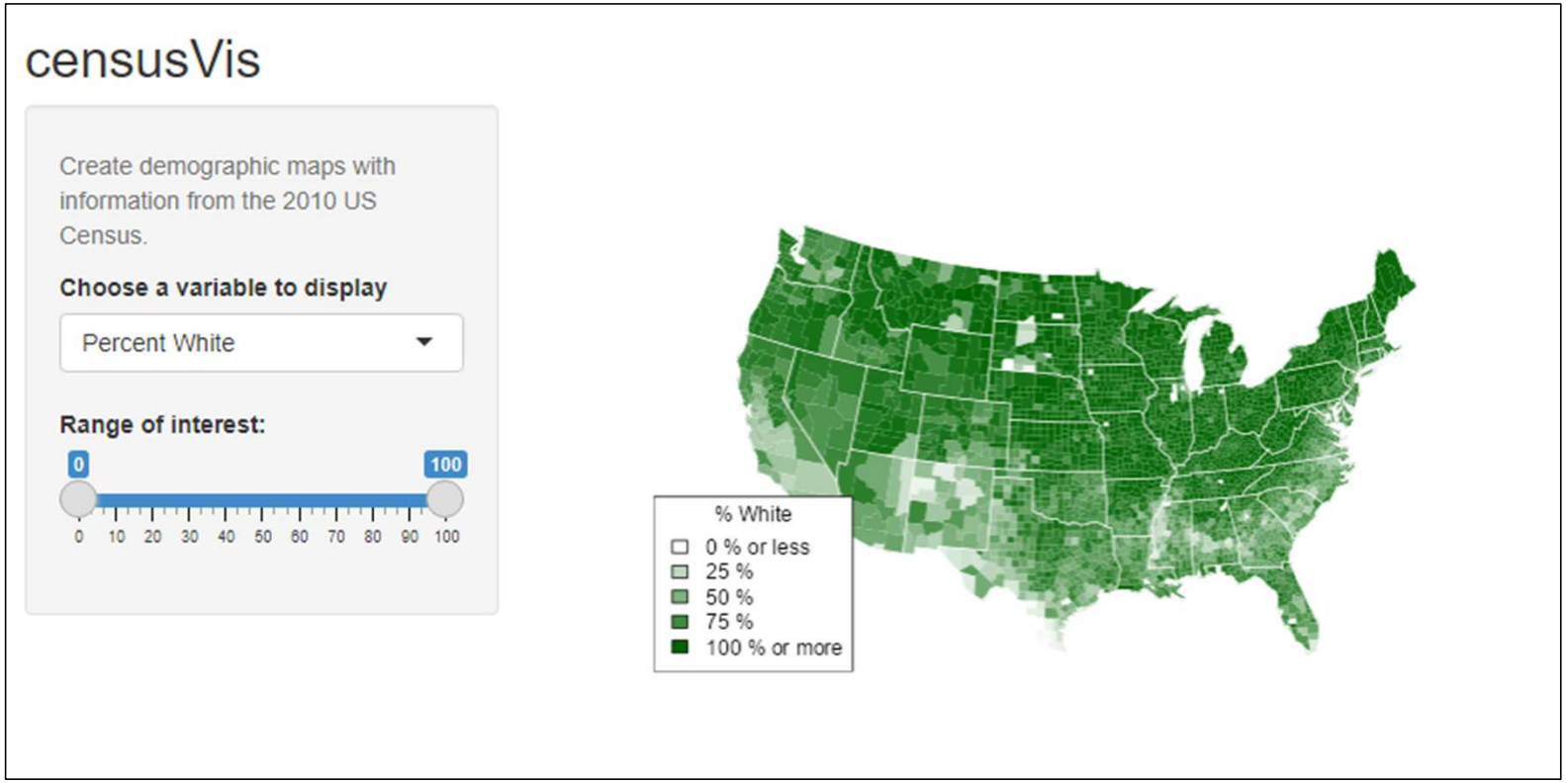
```
ui <- fluidPage(
  titlePanel("censusVis"),
  sidebarLayout(
    sidebarPanel(
      helpText("Create demographic maps with information from the 2010 US Census.")
      selectInput("var",
                  label = "Choose a variable to display",
                  choices = c("Percent White", "Percent Black", "Percent Hispanic", "Percent Asian"),
                  selected = "Percent White"),
sliderInput("range", label = "Range of interest:", min = 0, max = 100, value = c(0, 100))     ),
mainPanel(plotOutput("map"))   ) )
```

Use R scripts and data - server

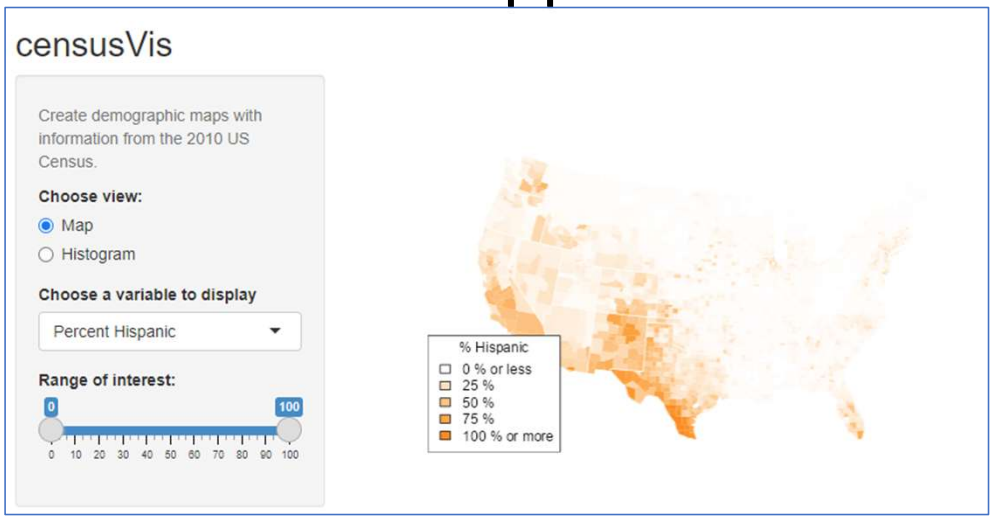
```
server <- function(input, output) {
  output$map <- renderPlot({
    data <- switch(input$var,   "Percent White" = counties$white,    "Percent Black" = counties$black,   "Percent Hispanic" = counties$hispanic,   "Percent Asian" = counties$asian)
      color <- switch(input$var,     "Percent White" = "darkgreen",    "Percent Black" = "black",    "Percent Hispanic" = "darkorange",   "Percent Asian" = "darkviolet")
      legend <- switch(input$var,   "Percent White" = "% White",   "Percent Black" = "% Black",   "Percent Hispanic" = "% Hispanic",    "Percent Asian" = "% Asian")
     percent_map(data, color, legend, input$range[1], input$range[2])   })
}
```

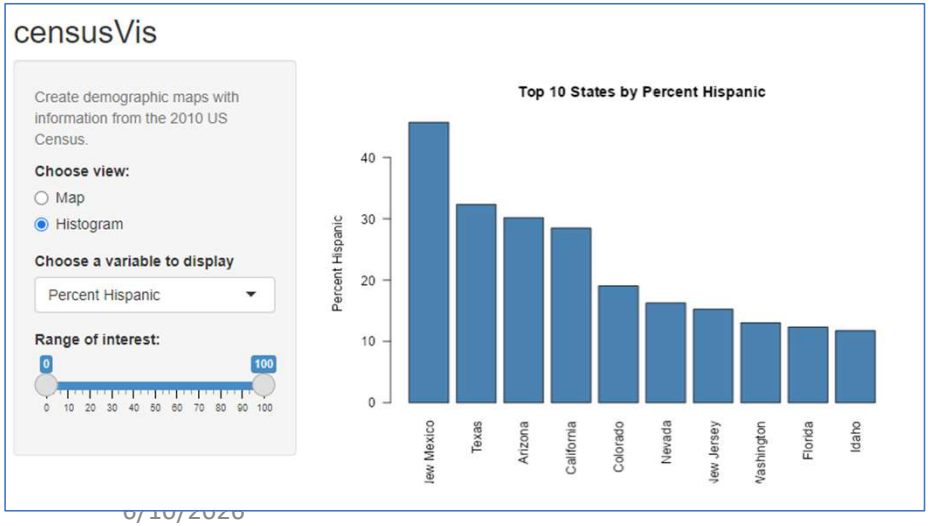
US census app

Source: https://shiny.rstudio.com/tutorial/written-tutorial/lesson5/



US census app





Which components are related to the following interaction steps?

1. Select – Mark something as interesting
2. Explore – Show me something else
3. Reconfigure – Show me a different arrangement
4. Encode – Show me a different representation
5. Abstract/Elaborate – Show me more or less detail
6. Filter – Show me something conditionally
7. Connect – Show me related items
8. Undo/Redo – Let me go to where I have already been
9. Change configuration – Let me adjust the interface

Reactive Expressions

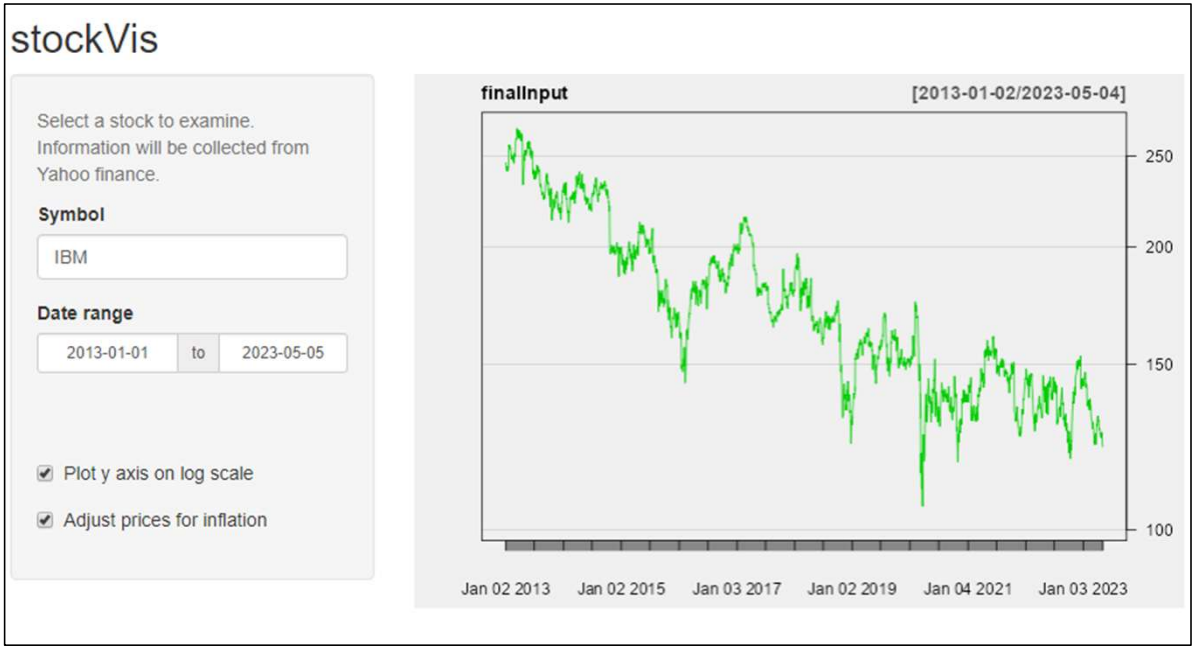
The stockVis app looks up stock prices by ticker symbol and displays the results as a line chart. The app lets you:

- Select a stock to examine
- Pick a range of dates to review
- Choose whether to plot stock prices or the log of the stock prices on the y axis, and
- Decide whether or not to correct prices for inflation.

Source: https://shiny.rstudio.com/tutorial/written-tutorial/lesson6/

Reactive Expressions

- By default, stockVis displays the SPY ticker (an index of the entire S&P 500).
- To look up a different stock, type in a stock symbol that Yahoo finance will recognize.
- You can look up Yahoo's stock symbols here. Some common symbols are GOOG (Google), AAPL (Apple), and GS (Goldman Sachs).



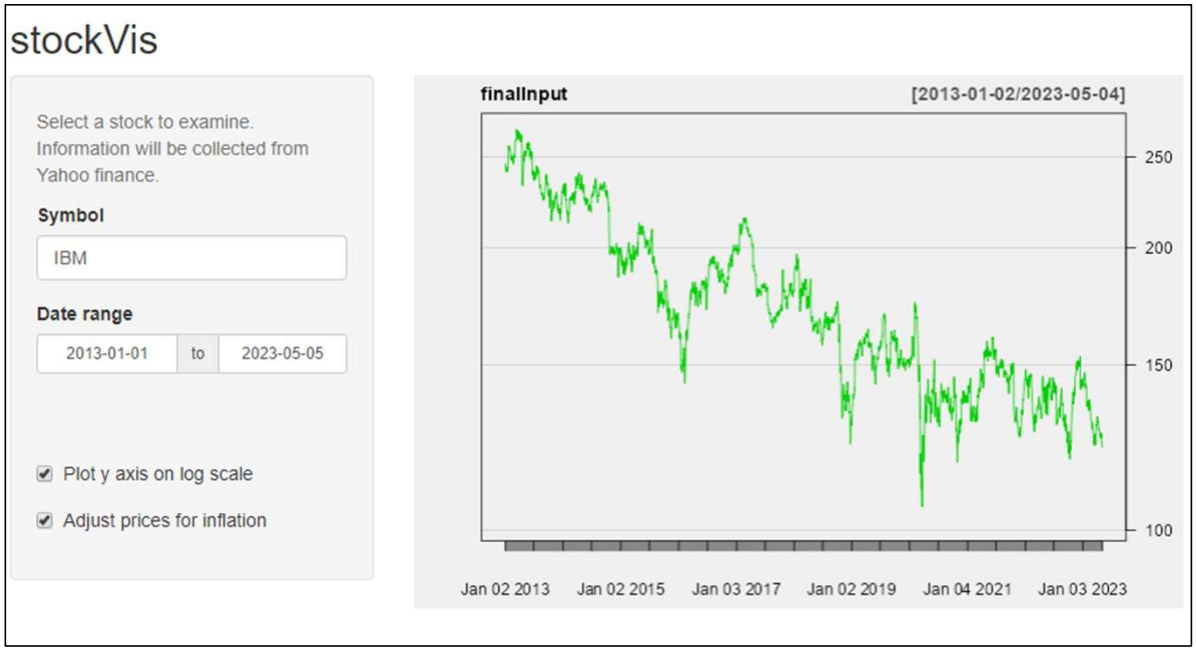
Stock vis

Source: https://shiny.rstudio.com/tutorial/written-tutorial/lesson6/

Reactive Expressions

stockVis relies heavily on two functions from the quantmod package:

- getSymbols to download financial data straight into R from websites like Yahoo finance and the Federal Reserve Bank of St. Louis.
- chartSeries to display prices in an attractive chart.
- stockVis also relies on an R script named helpers.R, which contains a function that adjusts stock prices for inflation.



Yahoo Finance: https://finance.yahoo.com

Stock vis

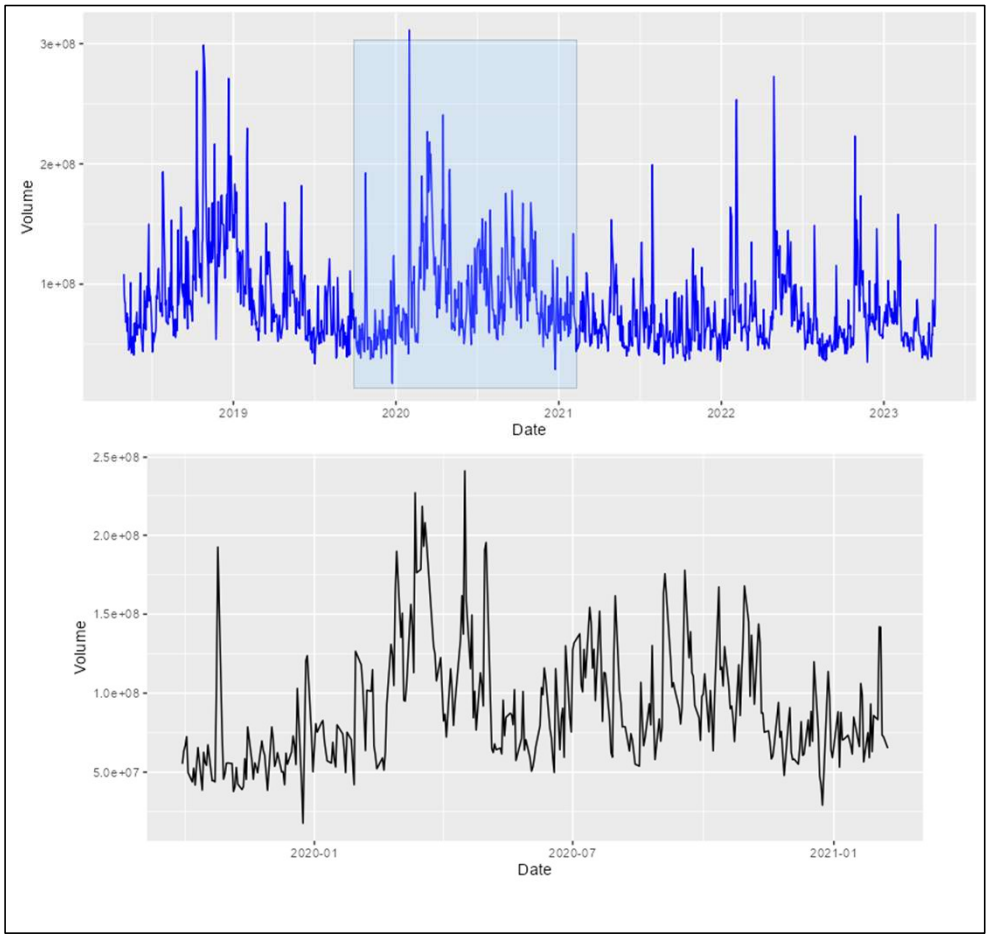
Brushing

- Brushing is the act of selecting a region or subset of data directly within a visualization.
- Typically done by clicking and dragging over a chart (e.g., time series or scatterplot).
- Helps users focus, filter, or zoom into specific parts of the data.

Why Brushing

- Enables Precise Selection
  - Lets users highlight specific data points or regions directly in the chart.
- Supports Focused Analysis – similar to drill down but more specific
  - Users can isolate subsets for closer inspection, comparison, or detail-on-demand.
- Facilitates Linked Views
  - Brushed data in one plot can dynamically update other visual elements (e.g., zoom, filters, summaries).

Brushing



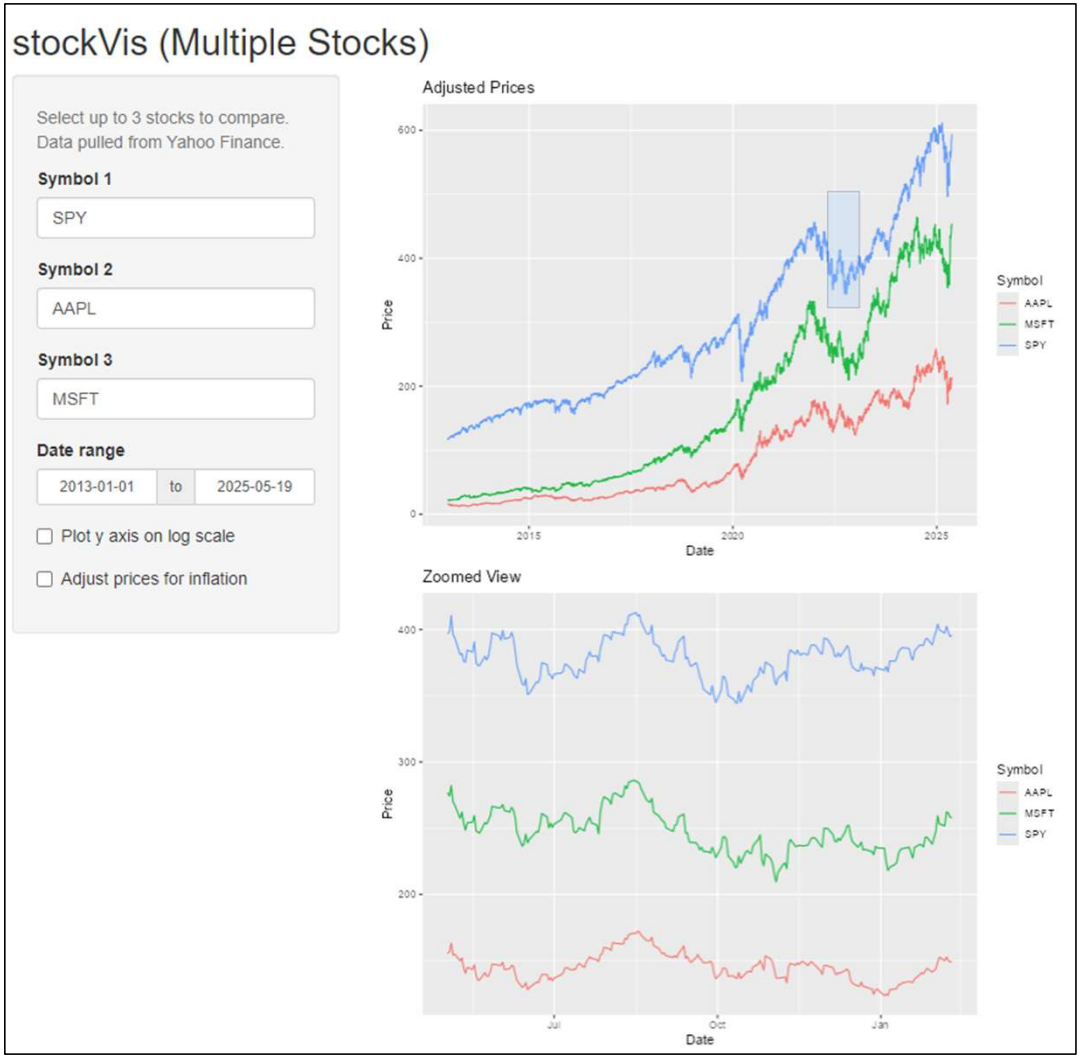
Brushing_1
Maxi_pattred…

Brushing

Which components are related to the following interaction steps?

1. Select – Mark something as interesting
2. Explore – Show me something else
3. Reconfigure – Show me a different arrangement
4. Encode – Show me a different representation
5. Abstract/Elaborate – Show me more or less detail
6. Filter – Show me something conditionally
7. Connect – Show me related items
8. Undo/Redo – Let me go to where I have already been
9. Change configuration – Let me adjust the interface

app_new_1.R



Assignment - new

- **Problem Definition**
  - Define a Data Science / analytics problem
  - Explain target users and decision-support goals
  - Explain how the interactive app helps solve the problem
- **Data Understanding**
  - Data source and structure
  - Exploratory analysis (trends, outliers, patterns)
  - Visual summaries and insights
- **Data Processing**
  - Cleaning, transformations, feature engineering
  - Explain why transformations were needed
- **Visualization & Interaction Design**
  - Explain visualization choices
  - Describe interactions and user flow
  - Justify design decisions
- **AI Usage Documentation (NEW)**
  - List AI tools used (ChatGPT, Claude, Copilot, etc.)
  - Document important prompts used
  - Specify which AI capabilities/skills were used:
    - Code generation
    - Debugging
    - UI design
    - Data analysis
    - Visualization suggestions
    - Refactoring
    - Documentation
    - Idea generation
  - Explain how AI outputs were validated/corrected
- **Evaluation & Reflection**
  - Strengths and limitations of the system
  - What AI helped with vs. human decisions
  - Lessons learned and future improvements
- **Deliverables**
  - Report
  - Source code
  - AI prompt appendix
  - Working demo (10–15 min presentation)